Statistical Language Model and Smoothing

Manish Shrivastava

Why are we doing classical?

- Rude Answer: Don’t know then *shrugging*
- Parrot Answer: Coz it shows historical journey of NLP
- SVM and Backpropogation Answer:
  - SVM algorithm was invented by Vladimir N. Vapnik and Alexey Ya. Chervonenkis in 1964. and finally used well in 1994+ (30 Years).
  - Backpropogation some way or another from 1950s in **modern day**. MLP with more than one layer - 1967.
- Sledgehammer or rock hammer?
  - We have multiple agents doing different jobs, when a query comes we usually route to one agent. If a compound query comes which has multiple questions or which needs multipl agents to generate a response then I should be able to identify each sub question route to the desired agent and concatenate the response.*

*Thanks Damodar for Example

Better Answer

- Starting of Modern NLP
- Learning LM teaches Atomic units which introduces notion of ,
  - Training, held out and Testing for NLP.
  - Perplexity metric (in evaluation lecture)
  - Sampling to generate sentences
  - Ideas like interpolation and backoff
  - As stated earlier not everything requires sledgehammer.

What to do with Tokens

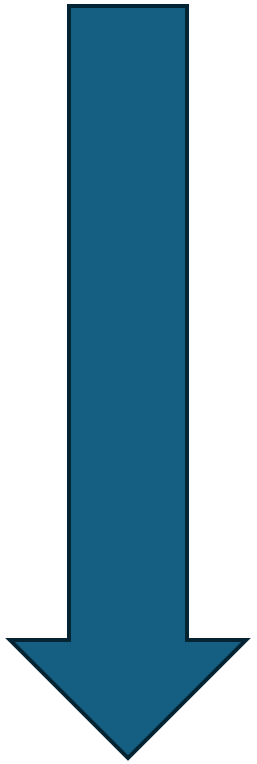
- Predict Next tokens? <- modern NLP (?) (Look at ANLP 2025)
- Language modelling:
  - Predicting Next token : Give probability distribution.
  - Assigning probability to a sentence(s?).
- Two threads
  - N-gram Models: This lecture
  - LLM: ANLP 2025/6

Prompt	sheldon : absolutely . but don ’ t be surprised *if*
Token, count	('worry', 4), ('you', 2), ('look', 1), ('punch', 1), ('expect', 1)
Actual Sentence:	Sheldon: Absolutely. But don’t be surprised if this movie sets you on the straight and narrow.
Probability (Bigram):	1.13E-34
Log Likelihood:	-78.16

Why do word prediction

- Query Autocomplete
- Spell Checking
- Grammar Checking
- Speech Recognition

- Rephrasing/Vibe-Coding



Easier & tangible Evaluation

Difficult & how to evaluate

LM Formally

- Shannon Game: Predict nth word give n-1 words
- Compute Probability of stream of tokens.
  - $P(W) = P(w_1,w_2,w_3,w_4,w_5 \ldots w_n)$
- Related: Find probability of new word being scanned.
  - $P(w_5 | w_1,w_2,w_3,w_4)$ or $P(w_n | w_1,w_2 \ldots w_{n-1})$
- Related: Find probability distribution of what could be next.

How to find these probabilities?

- Count and divide
- P(if | sheldon : absolutely . but don ’ t be surprised) =

$$\frac{C(\text{sheldon : absolutely . but don ’ t be surprised if})}{C(\text{sheldon : absolutely . but don ’ t be surprised})}$$

- Never see this occurrence again. Probability low.

- How much will one count?

Chain rule route

- Joint Probability
  - P(sheldon ,:, absolutely ,. ,but ,don, ’ t, be, surprised, if)
- Recall conditional Probabilities
  - $P(B|A) = P(A,B)/P(A)$ Rewriting: $P(A,B) = P(A)\,P(B|A)$
- More variables:

  $P(A,B,C,D) = P(A)\,P(B|A)\,P(C|A,B)\,P(D|A,B,C)$
- The Chain Rule in General

$$P(x_1,x_2,x_3,\ldots,x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1,x_2)\ldots P(x_n|x_1,\ldots,x_{n-1})$$

Chain rule to compute joint probability

$$P(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_{1:2})\ldots P(w_n|w_{1:n-1})$$
$$= \prod_{k=1}^{n} P(w_k|w_{1:k-1})$$

P(sheldon : absolutely . but don ’ t be surprised if) =

$$P(: \mid \text{sheldon}) \times$$
$$P(\text{absolutely} \mid \text{sheldon :}) \times$$
$$P(. \mid \text{sheldon : absolutely}) \times$$
$$P(\text{but} \mid \text{sheldon : absolutely .}) \times$$
$$P(\text{don't} \mid \text{sheldon : absolutely . but}) \times$$
$$P(\text{be} \mid \text{sheldon : absolutely . but don't}) \times$$
$$P(\text{surprised} \mid \text{sheldon : absolutely . but don't be}) \times$$
$$P(\text{if} \mid \text{sheldon : absolutely . but don't be surprised})$$

Markov Assumption

Andrei Markov

- Simplify

- P(if **|** sheldon : absolutely . but don ’ t be surprised)

  ≈ P(if|surprised)

$$P(w_n|w_{1:n-1}) \approx P(w_n|w_{n-1})$$

Unigram

$$P(w_1 w_2 \ldots w_n) \approx \prod_i P(w_i)$$

Bigram

$$P(w_{1:n}) \approx \prod_{k=1}^{n} P(w_k | w_{k-1})$$

- instead of:

$$\prod_{k=1}^{n} P(w_k | w_{1:k-1})$$

We approximate each component in the product

Trigram

$$P(w_i|w_1 w_2 \ldots w_{i-1}) \approx P(w_i|w_{i-2} w_{i-1})$$

General N-gram model

$$P(w_n|w_{1:n-1}) \approx P(w_n|w_{n-N+1:n-1})$$

5 different N-gram models

1-Gram	sheldon : is compwetewy you and still you wocka amy , ? to i : what two , sheldon just spent know we to engineering off re room : .
2-Gram	sheldon : who listens to , howard : and f . what you and we could you know , like to talk to a series of the gorillas .
3-Gram	sheldon : i don ’ t . uh , you know what will be my new lady friend who ’ s where his little girl , and i was humiliated by yet
4-Gram	sheldon : nothing . not a good time to evaluate the beta test and see where we stand ? penny : why do you keep assuming it was me .
5-Gram	sheldon : oh , don ’ t wake up tomorrow morning . leonard : did something happen today that ’ s bothering you ? leonard : oh , yay for me .

Last two make use of backoff

Problems with N-gram models

1. N-grams can't handle **long-distance dependencies**:

"**The soups** that I made from that new cookbook I bought yesterday **were** amazingly delicious."

2. N-grams don't do well at modeling new sequences (even if they have similar meanings to sequences they've seen)

The solution: **Large language models**

- can handle much longer contexts
- because they use neural **embedding spaces**, can model meaning better

Reliability v/s Discrimination

“large green _____”

tree? mountain? frog? car?

“swallowed the large green _____”

pill? candy?

Reliability v/s Discrimination

- **larger n: more information about the context of the specific instance (greater discrimination)**

- **smaller n: more instances in training data, better statistical estimates (more reliability)**